```
the:  3
count:  2
word:  2
sounds:  2
```

In this example, each message returns a new collection of data that is processed by the next message until we finally return the list of strings that is sent to puts.

You may wonder how you might debug that long chain of methods if something is not working and you want to determine what each individual step is. Ruby provides a method called tap that is designed to allow you to "tap into" this kind of method pipeline. All tap does is take a block, pass the receiver into the block, and then return the original receiver of the method (which, from the perspective of the method pipeline does nothing—the receiver calls tap and then the same object is returned to receive the next method in the chain). So tap is a no-op, except that it does invoke a block. That block could have a side effect, such as printing a value to the console for debugging purposes:

tut_containers/word_freq/bester_word_count_with_tap.rb

```
require_relative "words_from_string"

raw_text = "The problem breaks down into two parts. First, given some text
as a string, return a list of words. That sounds like an array. Then, build
a count for each distinct word. That sounds like a use for a hash---we can
index it with the word and use the corresponding entry to keep a count."

puts words_from_string(raw_text)
  .tally
  .sort_by { |word, count| count }
  .tap { |result| puts "sorted tally: #{result}\n\n" }
  .last(5)
  .tap { |result| puts "only the last five: #{result}\n\n" }
  .reverse
  .tap { |result| puts "reversed: #{result}\n\n" }
  .map { |word, count| "#{word}:  #{count}" }
```

produces:

```
sorted tally: [["keep", 1], ["problem", 1], ["breaks", 1], ["down", 1], ["into",
1], ["two", 1], ["parts", 1], ["first", 1], ["given", 1], ["some", 1], ["text",
1], ["as", 1], ["we", 1], ["can", 1], ["index", 1], ["it", 1], ["with", 1],
["and", 1], ["corresponding", 1], ["entry", 1], ["to", 1], ["string", 1],
["return", 1], ["list", 1], ["of", 1], ["words", 1], ["an", 1], ["array", 1],
["then", 1], ["build", 1], ["each", 1], ["distinct", 1], ["hash", 1], ["like",
2], ["for", 2], ["use", 2], ["that", 2], ["sounds", 2], ["word", 2], ["count",
2], ["the", 3], ["a", 6]]

only the last five: [["sounds", 2], ["word", 2], ["count", 2], ["the", 3], ["a",
6]]

reversed: [["a", 6], ["the", 3], ["count", 2], ["word", 2], ["sounds", 2]]

a:  6
the:  3
count:  2
word:  2
sounds:  2
```